CM-1 Introduction to data analysis and python tools

3TLDE.A13 - Législation Numérique et Enjeux de l'Analyse des Données

27 April 2026

Planing of the course

1. CM-1 + TD-1: Introduction to Data Analysis and Python Tools
2. CM-2 + TD-2: Data Cleaning and Preprocessing
3. CM-3 + TD-3: Exploratory Data Analysis and Visualization
4. CM-4 + TD-4: Feature Analysis, Correlation, and Dimensionality Reduction
5. CM-5 + CM-6: From Data Analysis to Machine Learning
6. TD-5: Evaluation

Evalustions

- QCM-1, 10 minutes at the end of TD-2 (4 may 2026), 15%
- QCM-2, 10 minutes at the end of TD-4 (1 juin 2026), 15%
- Coding: 1.5h at TD-5 (3 juin 2026), 30%
- Written exam: 1h at TD-5 (3 juin 2026), 40%

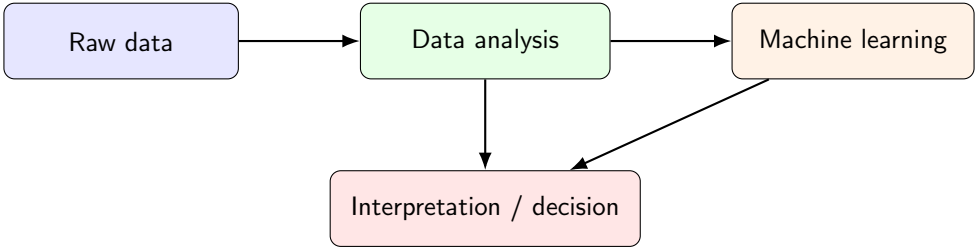
Why study data analysis?

- Modern systems produce **tables, logs, signals, texts, images, and events**.
- Before building predictive models, we must **understand the data**.
- Data analysis helps answer practical questions:
  - What do we have?
  - Is the data reliable?
  - What patterns appear?
  - Which variables may be useful later for machine learning?
- In industry, good analysis often prevents expensive mistakes.

Core idea

Data analysis is not only about drawing plots. It is a process of **understanding, checking, summarizing, and preparing data for decisions or models**.

From this course to machine learning



- This course builds the **foundation**.
- Later courses will focus on **prediction**, **optimization**, and **machine learning**.
- Without understanding the dataset, model performance is often misleading.

Learning objectives of CM-1

At the end of this lecture, you should be able to:

- explain the main steps of a simple data analysis workflow;
- distinguish **data**, **information**, and **knowledge**;
- recognize different data types and common analytical tasks;
- understand the role of **Python** in data analysis;
- use the main libraries: `NumPy`, `pandas`, and `seaborn`;
- load and inspect a dataset with Python;
- perform simple summaries on the Titanic dataset.

Note: you can find the code for CM-1 on MOOTSE.

Table des matières

Data, information, and knowledge

Data

Raw observations

- age = 22
- fare = 7.25
- class = 3

Information

Organized meaning

- average age
- missing values
- class distribution

Knowledge

Useful understanding

- survival differs by class
- some variables need cleaning
- patterns may guide modeling

Role of data analysis

Transform **raw data** into **structured information** and then into **actionable understanding**.

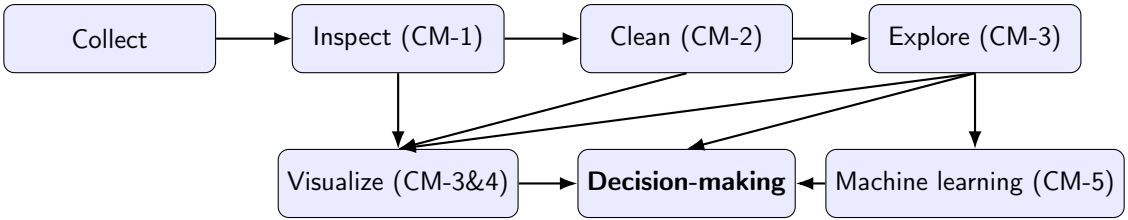
Typical questions in data analysis

- How many observations are available?
- Which variables are numerical? Which are categorical?
- Are there missing, duplicated, or inconsistent values?
- What is the scale of each feature?
- Are some variables strongly related?
- Do groups behave differently?
- Is the dataset ready for machine learning?

Important mindset

A data analyst does not start by applying complicated algorithms. The first task is to **understand the structure and quality of the data**.

A simple workflow of data analysis



- The process is often **iterative**, not strictly linear.
- New anomalies discovered during exploration may force us to return to cleaning.

Common types of data

Structured data

- rows and columns, tables, spreadsheets, SQL
- easiest format for introductory analysis

Unstructured data

- text, images, audio, video
- richer but harder to analyze directly

Semi-structured data

- JSON, XML, logs
- flexible schema

Time-dependent data

- time series, event streams
- ordering matters

In this first course, we mainly work with **structured tabular data**.

Types of variables in tabular data

Numerical variables

- continuous: age, salary, temperature
- discrete: number of children, number of purchases

Categorical variables

- nominal: color, city, sex
- ordinal: low, medium, high

Why does this matter?

The type of a variable influences:

- the summaries we compute;
- the plots we choose;
- the preprocessing needed for machine learning.

Examples of analytical tasks

1. **Descriptive analysis**: summarize what is in the data.
2. **Diagnostic analysis**: explain why something happened.
3. **Predictive analysis**: estimate what may happen next.
4. **Prescriptive analysis**: support decisions or actions.

In this course

We focus first on **descriptive** and **exploratory** analysis, because these are the basis for later predictive modeling.

Descriptive analysis vs exploratory analysis

Descriptive analysis

- summarize known quantities
- counts, means, percentages
- often answers precise questions

Exploratory analysis

- search for patterns or anomalies
- visualization and group comparison
- often produces new questions

Both are important. In practice, analysts move back and forth between them.

Table des matières

Why Python?

- simple syntax, readable code;
- large ecosystem for scientific computing;
- widely used in different domains;
- easy transition from analysis to machine learning and deep learning.

The main Python tools in this course

Library	Main role
`NumPy`	numerical arrays, vectorized operations, linear algebra
`pandas`	tabular data, loading files, filtering, grouping
`matplotlib`	basic plotting and customization
`seaborn`	statistical visualization with simple syntax
`scikit-learn`	preprocessing and machine learning tools
`Anaconda`	integrated full-stack tools, package management, and virtual environment functionality
`jupyter notebook`	interactive environment for code, text, and outputs
...	...

Notebook or script?

Notebook

- interactive
- ideal for experiments
- combines code, text, and figures
- excellent for teaching and prototyping

Python script

- reproducible workflow
- easier to reuse in projects
- suitable for larger programs
- better for software organization

NumPy: arrays instead of simple lists

- A Python list is flexible but not ideal for large numerical computations.
- A `NumPy` array stores numerical data efficiently.
- Many operations can be applied to the whole array at once.

```
1 import numpy as np
2 x = np.array([1, 2, 3, 4])
3 print(x.mean())
4 print(x * 2)
```

Main idea

Instead of looping element by element, we often use **vectorized operations** (broadcast).

pandas: the central tool for tabular data

- The main object is the `DataFrame`.
- A DataFrame is a table with row indices and column names.
- Columns may contain different types of values.

```python
import pandas as pd

df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Clara'],
    'age': [21, 25, 22],
    'score': [14.5, 12.0, 16.0]
})
print(df)
```

matplotlib and seaborn

matplotlib

- low-level plotting library
- very flexible
- many options for customization

seaborn

- built on top of matplotlib
- easier statistical plots
- cleaner defaults for quick EDA

Examples of plots we will use later:

- histogram;
- boxplot;
- scatter plot;
- count plot;
- heatmap.

scikit-learn: beyond modeling

Except for machine learning models, `scikit-learn` is also useful for:

- train-test splitting;
- scaling and normalization;
- encoding categorical variables;
- pipelines;
- dimensionality reduction;
- evaluation metrics.

Note

In this course, `scikit-learn` will appear progressively, first for **preprocessing**, then for simple learning tasks.

A typical Python import block

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

- `np`, `pd`, `plt`, and `sns` are common aliases.
- Using standard import conventions makes code easier to read.

Table des matières

Variables and basic data types

```
1 age = 22
2 name = "Alice"
3 height = 1.68
4 is_student = True
```

Common basic types:

- `int`
- `float`
- `str`
- `bool`

In data analysis, variables in Python store values, but variables in a dataset represent **features or attributes**.

Lists, dictionaries, and tuples

```python
ages = [21, 25, 22]
student = {
    'name': 'Alice',
    'age': 21,
    'city': 'Lyon'
}
point = (3, 5)
```

- list: ordered and mutable
- dictionary: key-value structure
- tuple: ordered and immutable

These structures are useful, but for real tabular data, `pandas` is usually more convenient.

Indexing and slicing of list

```
1 x = [10, 20, 30, 40, 50]
2 print(x[0])      # first element
3 print(x[-1])     # last element
4 print(x[1:4])    # slice, x[1], x[2], x[3]
```

Why important?

Many data structures in Python, NumPy, and pandas rely on indexing to select values, rows, or columns.

Conditions and loops

```python
fare = 35
if fare > 30:
    print("expensive")
else:
    print("cheap")

print("expensive") if fare > 30 else print("cheap")

for x in [1, 2, 3]:
    print(x)
```

- conditions control logic;
- loops repeat instructions;
- in data analysis, many explicit loops can be replaced by pandas or NumPy operations.

Functions

```python
def describe_age(age):
    if age < 18:
        return "minor"
    elif age < 60:
        return "adult"
    else:
        return "senior"

describe_age(30)
```

- Functions organize code into reusable blocks.
- We will write small functions for cleaning or transforming data.

Classes and Objects

```python
class Model:
    def __init__(self, name):
        self.name = name

    def predict(self, x):
        return f"{self.name} predicts {x}"

m = Model("LinearReg")
print(m.predict(10))
```

- **Class**: a blueprint for creating objects
- **Object**: an instance of a class
- `self`: refers to the current instance

Why important in Data Science?

Most libraries like `scikit-learn` or `PyTorch` use classes to define models, estimators, and custom data transformers.

Reading error messages

- Programming errors are normal.
- A good analyst first learns to read the last line of the traceback.
- Common beginner mistakes:
  - missing parenthesis;
  - wrong indentation;
  - typo in a variable or column name;
  - applying a numerical operation to strings.

Best habit

When code fails, do not only try random fixes.
First ask: **What object do I have? What type is it? What does the error tell me?**

Reading documentation

- You cannot (and should not) memorize every parameter of every function.
- A good analyst knows how to find help quickly.

Ways to get help in Python

```python
import pandas as pd

# 1. Use the help function
help(pd.read_csv)

# 2. In Jupyter, use the question mark
pd.read_csv?

# 3. Official online, search with names of classes or functions
```

What to look for?

- **Parameters**: What inputs does the function expect?
- **Return value**: What does the function give back?
- **Examples**: Usually at the bottom, they are the fastest way to learn.

Table des matières

Pandas: creating a DataFrame

```python
import pandas as pd

df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Clara'],
    'age': [21, 25, 22],
    'city': ['Lyon', 'Paris', 'Lille']
})
```

- each key becomes a column;
- each list contains the values of that column.

Pandas: loading a CSV file

```python
import pandas as pd

df = pd.read_csv('titanic.csv')
```

Common file-reading functions:

- `pd.read_csv(...)`
- `pd.read_excel(...)`
- `pd.read_json(...)`

In practice

The first real step after loading is not modeling. It is **inspection**.

Pandas: first inspection commands

```python
print(df.head())
print(df.shape)
print(df.columns)
print(df.dtypes)
print(df.info())
```

These commands help answer:

- How many rows and columns?
- What are the variable names?
- Which types are recognized?
- Are there missing values?

Pandas: selecting columns and rows

```
ages = df['Age']
subset = df[['Age', 'Sex']]
first_rows = df.iloc[:5]
first_row = df.iloc[0]
```

- `df['Age']` selects one column.
- `df[['Age', 'Sex']]` selects several columns.
- `iloc` uses integer position index.

Pandas: filtering rows

```python
adults = df[df['Age'] >= 18]
women = df[df['Sex'] == 'female']
rich_passengers = df[df['Fare'] > 100]
```

- Filtering keeps only rows satisfying a condition.
- This is fundamental for group comparison and targeted exploration.

Pandas: simple summaries with pandas

```
print(df['Age'].mean())
print(df['Sex'].value_counts())
print(df['Fare'].describe())
```

Typical summary methods:

- `mean()`, `median()`, `std()`
- `min()`, `max()`
- `value_counts()`
- `describe()`

Pandas: grouping data

```
1 print(df.groupby('Sex')['Survived'].mean())
2 print(df.groupby('Pclass')['Fare'].mean())
```

- groupby is one of the most useful tools in pandas.
- It allows us to compare statistics by group.

Example question

Did survival rate differ between male and female passengers?

Pandas: handling missing values

```python
print(df.isnull().sum())

df_age_filled = df['Age'].fillna(df['Age'].mean())

df_clean = df.dropna(subset=['Embarked'])
```

- `isnull().sum()` counts missing values by column.
- `fillna(...)` replaces missing values.
- `dropna(...)` removes rows with missing values.

In practice

Missing values must be checked early because they may affect summaries, plots, and later models. We will see it in detail during CM-2.

Pandas: sorting and creating a new column

```
df_sorted = df.sort_values(by='Fare', ascending=False)

df['IsChild'] = df['Age'] < 18

print(df[['Age', 'IsChild']].head())
```

- `sort_values(...)` orders rows by a variable.
- New columns can be created from existing ones.
- Feature creation is a common step in data analysis.

Why do we also need NumPy?

- `pandas` is very convenient for tabular data.
- `NumPy` is the core library for numerical computation in Python.
- It provides fast multidimensional arrays and vectorized operations.
- Many scientific libraries, including `pandas`, rely on NumPy internally.

Idea

Use `pandas` for structured tables, and `NumPy` for efficient numerical operations.

NumPy: creating arrays

```python
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([[1, 2], [3, 4]])
c = np.arange(0, 10, 2)

zeros = np.zeros((2, 3))
ones = np.ones((3, 2))

same_zeros = np.zeros_like(a)
same_ones = np.ones_like(b)
```

- `np.array(...)` creates an array from Python lists.
- `zeros(...)` and `ones(...)` create arrays with fixed values.
- `arange(start, stop, step)` creates a sequence of values.
- `zeros_like(...)` and `ones_like(...)` create arrays with the same shape as an existing one.

NumPy: arange vs linspace

```python
import numpy as np

a = np.arange(0, 10, 2)
b = np.linspace(0, 10, 6)

print(a)
print(b)
```

- `arange(start, stop, step)` uses a fixed step.
- `linspace(start, stop, n)` creates a fixed number of values.
- Both are useful, but for different purposes.

NumPy: shape, indexing, and slicing

```python
import numpy as np

b = np.array([[1, 2], [3, 4], [5, 6]])
a = np.array([10, 20, 30, 40, 50, 60])

print(b.shape)
print(b[0, 1])
print(b[:, 0])
print(b[1:])

print(a[::2])
print(a[1::2])
print(a[::-1])
```

- `shape` gives the dimensions of the array.
- `b[0,1]` accesses one element.
- `b[:,0]` selects the first column.
- Slicing follows the form `start:stop:step`.
- For example, `a[::2]` keeps one element every two positions.

NumPy: vectorized operations

```python
import numpy as np

a = np.array([1, 2, 3, 4])

print(a + 10)
print(a * 2)
print(a ** 2)

mask = a > 2
print(mask)
print(a[mask])
```

- Operations are applied to all elements at once.
- This is called **vectorization**.
- Boolean masks are useful for filtering values.

NumPy: basic statistics

```python
import numpy as np

a = np.array([4, 7, 2, 9, 5])

print(np.mean(a))
print(np.std(a))
print(np.min(a))
print(np.max(a))
print(np.sum(a))

# equivalent to a.fuction_name()
```

Common numerical functions:

- `mean()`, `std()`
- `min()`, `max()`
- `sum()`

NumPy: generating random values

```python
import numpy as np

np.random.seed(42)

x = np.random.randint(0, 10, size=5)
y = np.random.rand(5)

print(x)
print(y)
```

- Random arrays are useful for experiments and simulations.
- `seed(...)` makes results reproducible.

Why Seaborn for data visualization?

- `seaborn` is a high-level library for statistical graphics.
- It works very well with `pandas` DataFrames.
- It makes common plots easier to create and easier to read.

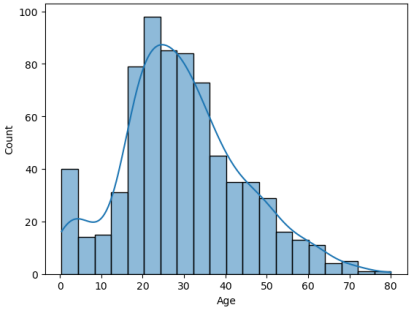
Goal

Visualizations help us detect distributions, outliers, group differences, and possible relationships between variables.

Seaborn: histogram of a numerical variable

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(data=df, x='Age', bins=20, kde=True)
plt.show()
```

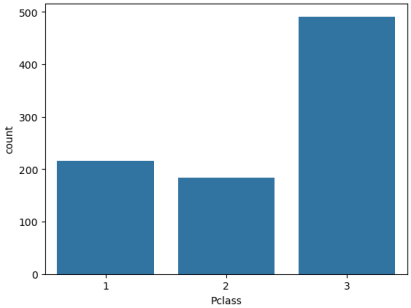


- A histogram shows the distribution of a numerical variable.
- Here we inspect the ages of Titanic passengers.
- The KDE curve gives a smoothed view of the distribution.

Seaborn: countplot for a categorical variable

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(data=df, x='Pclass')
plt.show()
```

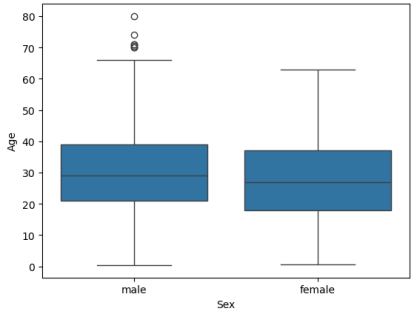


- A countplot shows how many observations belong to each category.
- It is useful for categorical variables such as class or sex.

Seaborn: boxplot for group comparison

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(data=df, x='Sex', y='Age')
plt.show()
```

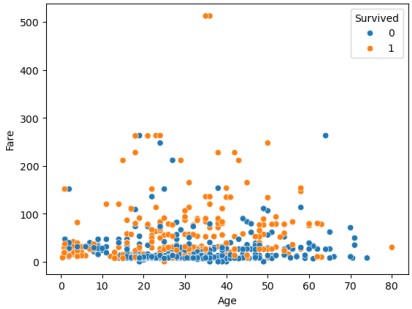


- A boxplot summarizes median, quartiles, and possible outliers.
- It is useful to compare a numerical variable across groups.

Seaborn: scatterplot for relationships

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(data=df, x='Age', y='Fare', hue='Survived')
plt.show()
```



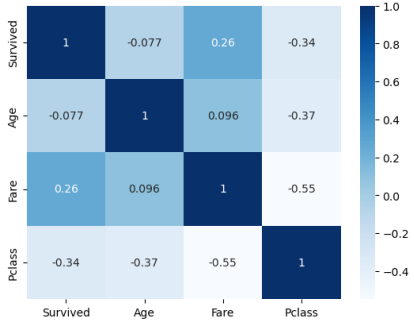
- A scatterplot helps visualize relationships between two numerical variables.
- The color can represent an additional categorical variable.

Seaborn: correlation heatmap

```python
import seaborn as sns
import matplotlib.pyplot as plt

corr = df[['Survived', 'Age', 'Fare', 'Pclass']].corr()

sns.heatmap(corr, annot=True, cmap='Blues')
plt.show()
```



- A heatmap gives a compact view of correlations.
- It helps identify variables that may be related.

Table des matières

Good habits from the beginning

- Keep raw data unchanged.
- Use clear variable names.
- Comment only when the intention is not obvious.
- Check shapes, types, and missing values early.
- Save important intermediate results.
- Make your workflow reproducible.

Common beginner mistakes

- jumping directly to modeling;
- ignoring missing values;
- confusing rows and columns;
- mixing strings and numbers in one column;
- trusting plots or summaries without checking the underlying data;
- overwriting the original DataFrame without caution.

Reproducibility matters

- If analysis cannot be repeated, it is difficult to trust.
- Reproducibility means:
  - same data source;
  - same code;
  - same transformations;
  - same interpretation process.
- Later, this will become even more important for machine learning experiments.

A minimal analysis template

```python
import pandas as pd

# 1. Load
df = pd.read_csv('titanic.csv')

# 2. Inspect
print(df.head())
print(df.info())

# 3. Summarize
print(df.describe(include='all'))

# 4. Explore a question
print(df.groupby('Sex')['Survived'].mean())
```

Takeaways

1. Data analysis is a process of understanding and preparing data.
2. Python is a practical ecosystem for modern data analysis.
3. `pandas`, `NumPy` and `seaborn` are essential tools for data analysis.
4. First inspection steps are important: shape, columns, types, and missing values.
5. Good habits now will help for machine learning later.

Preview of the next lecture

CM-2: Data Cleaning and Preprocessing

Topics:

- missing data;
- duplicates and inconsistencies;
- encoding categorical features;
- scaling and normalization;
- preparing data for machine learning.